Probabilistic analysis solutions

05-E01 Expectations add despite dependence

Let I_1, I_2 indicate sixes. Each marginal mean is 1/6, so E[X]=1/3 without independence. Exactly one six is the union of two disjoint events; independence factors each intersection, giving (1/6)(5/6)+(5/6)(1/6)=5/18. If the second die copies the first, marginal means and E[X]=1/3 remain unchanged, but X is only 0 or 2, making P(X=1)=0. Dependence changes the joint law without invalidating linearity.

05-E02 Fixed points of a permutation

A specified badge is fixed in (n−1)! permutations, probability 1/n. Summing n indicators gives E[F]=1. Two specified fixed positions leave (n−2)! permutations, giving $1/[n(n-1)]$, unequal to marginal product $1/n^2$. The events are dependent. At n=1 the only permutation has F=1; at n=0 the empty permutation has F=0. The two-position formula only applies to $n \geq 2$. At n=2, F=0 and F=2 each have probability 1/2, illustrating why expectation 1 is not certainty of one fixed point.

05-E03 Six reverse Fisher–Yates paths

j_2	j_1	Output
0	0	[b,c,a]
0	1	[c,b,a]
1	0	[c,a,b]
1	1	[a,c,b]
2	0	[b,a,c]
2	1	[a,b,c]

Every path has probability (1/3)(1/2)=1/6. Generally, before round i the suffix i+1..n−1 is fixed. Conditional on any fixed suffix, each of the i+1 remaining identities is selected for position i with probability 1/(i+1), and that position is never changed again. A specified complete permutation has one legal choice at each stage, multiplying to (1/n)(1/(n−1))…(1/2)=1/n!. This proves the full joint law, not merely position marginals, and is the reverse counterpart of the forward proof.

05-E04 Expected messages from random splits

Each split preserves p+(n−p)=n. All terminal sizes equal 1, so every tree has exactly n leaves and L_n=n. Strong induction in $L_n = 2\sum_{j<n} L_j/(n-1)$,$L_1$=1 gives the same result.

Probabilistic analysis solutions (continued)

For stars, $S_1=0$ and $S_n = n + 2\sum_{j<n} S_j/(n-1)$ by conditioning on p. Multiplying by n−1 and subtracting the n equation from the n+1 equation yields

$$nS_{n+1} - (n-1)S_n = 2n + 2S_n, \quad S_{n+1} = 2 + \frac{n+1}{n}S_n.$$

For $Q_n=S_n/n$, the difference is 2/(n+1), with $Q_1=0$. Summing gives $Q_n=2(H_n-1)$, hence $S_n=2n(H_n-1)$. At n=5, every execution prints five exclamation marks; expected stars equal 77/6. Each execution prints an integer number of characters, while their average may be fractional.

05-E05 Exactly two hires

Exactly one hire means the global best is first, probability 1/n. Exactly n hires requires the sole increasing permutation, probability 1/n!. For exactly two hires, the second hire must be the global best. If it arrives at $i \geq 2$, the first i−1 candidates must have their maximum in position 1, so there was only the first hire before it. Conditional on the best at i, earlier relative order is uniform, giving probability 1/(i−1). Disjoint best-position cases give

$$P(X=2) = \sum_{i=2}^{n} \frac{1}{n(i-1)} = H_{n-1}/n.$$

For n=6 the three answers are 1/6,137/360,1/720. At n=1, “one” and “n” hires are the same event, not disjoint categories; two hires are impossible.

05-E06 Marginals and random-path counts

Each of four offsets has probability 1/4. For a fixed identity and target position exactly one offset maps between them, proving uniform marginals. Only four cyclic shifts occur; the other twenty permutations have probability zero rather than 1/24.

The full-range swap algorithm on three elements has 27 equiprobable paths. Output probabilities are integer multiples of 1/27 and cannot all equal 1/6. In the correct shuffle, the processed prefix is never disturbed. Full-range later swaps can revisit it, destroying the invariant. These arguments concern the joint distribution; an individual-position histogram or a visually mixed result cannot refute them.

05-E07 Bias from priority ties

Among sixteen equiprobable pairs, six have $P_1<P_2$, four tie, and six have $P_1>P_2$. Stability gives original order probability 10/16=5/8 and reverse probability 6/16=3/8.

Probabilistic analysis solutions (continued)

Redrawing ties accepts with probability 12/16=3/4. Conditional on acceptance, six pairs support each order, giving one half. The final probability is also $\sum_{r\ge 0}(1/4)^r(6/16)=1/2$. Expected rounds are 1/(3/4)=4/3. A high probability of no ties is not unconditional exact uniformity.

05-E08 Nonuniform birthdays

Decompose the pair collision into disjoint common-date events. Independence gives $q=\sum_r p_r^2$, and linearity gives $E[X]=\binom{k}{2}q$, without independent pair indicators. For n dates,

$$\sum_r (p_r-1/n)^2 = q-1/n \ge 0.$$

Equality holds exactly for all p_r=1/n, proving uniform minimization.

Here q=1/4+1/16+1/64+1/64=11/32, and E[X]=33/32 for three people. Distinct dates have probability

$$6\left(\frac{1}{64}+\frac{1}{64}+\frac{1}{128}+\frac{1}{256}\right)=33/128,$$

so any collision has probability 95/128. Expectation above 1 is not a probability. The uniform complement product does not apply because conditional remaining probability mass depends on which dates have appeared.

05-E09 Four balls, three bins, and collection

A bin is empty with probability $(2/3)^4$, so E[empty]=3 • 16/81=16/27. Its singleton probability is $\binom{4}{1}(1/3)(2/3)^3$; summing three bins gives 32/27. Inclusion–exclusion counts surjective assignments of four named balls:

$$P(\text{all occupied})=(3^4-3\cdot 2^4+3\cdot 1^4)/3^4=4/9.$$

For random completion from zero balls, new-bin success probabilities are 1,2/3,1/3, giving expected waits 1,3/2,3 and total 11/2. This random total differs from the fixed-four-throw experiment.

05-E10 Overlapping windows are dependent

There are six length-3 windows, each all H with probability 1/8, so E[X]=3/4. Count prefixes without HHH by tail-length states (a,b,c) for 0,1,2 consecutive Hs. Appending T maps all states to 0; H maps 0→1 and 1→2, while H from 2 is forbidden. The update is (a+b+c,a,b).

Probabilistic analysis solutions (continued)

Flips	a	b	c	Valid prefixes
0	1	0	0	1
1	1	1	0	2
2	2	1	1	4
3	4	2	1	7
4	7	4	2	13
5	13	7	4	24
6	24	13	7	44
7	44	24	13	81
8	81	44	24	149

Every valid new prefix has a unique predecessor and last symbol, proving no omission or duplication. Of 256 equally likely strings, 149 avoid HHH; thus P(L≥3)=107/256, not 3/4. A length-4 H run contributes two windows to X, while the event only asks whether at least one exists.

05-E11 The optimal threshold for seven candidates

Conditional on the best at i>k, success requires the maximum of its i−1 predecessors among the first k, with probability k/(i−1). Position i has probability 1/7. Thus $P_k = (k/7)\sum_{j=k}^{6} 1/j$. Writing that sum as q_k, the adjacent difference is $(q_{k+1}-1)/7$; q decreases strictly. Since $q_2=29/20>1$ and $q_3=19/20<1$, optimal k=2 and $P_2=29/70$. For k=1,3,4,5,6, probabilities are 7/20,57/140,37/105,11/42,1/7, all smaller.

In the specified trace, the first two set M=3. Candidate 6 at position 3 is the first exceedance and is selected, but candidate 7 appears at position 6. This execution fails. The strategy optimizes uniform-order success probability, not success on every order.

05-E12 Preserve the Quicksort model

The interval $z_2..z_5$ has four keys. Its first pivot must be an endpoint for a direct comparison, giving 2/4=1/2. The six-key interval $z_1..z_6$ similarly gives 2/6=1/3. Summing all pair indicators gives $2(7)H_6 - 24 = 14 \cdot 49/20 - 24 = 103/10$.

Probabilistic analysis solutions (continued)

With all equal keys and the specified two-way partition, every pivot gives sizes n−1 and 0. Thus C(n)=C(n−1)+n−1,C_1=0, solving to n(n−1)/2, or 15 at n=6. Choosing a different equal identity does not alter the recurrence. The distinct-key expectation is 103/10; equal keys take fixed 15 and remain Θ(n^2) asymptotically. Three-way equality handling changes the model.

05-E13 Six outcomes from fair bits

Draw three bits as z in 0..7. Return z+1 when z≤5; otherwise retry. Each legal result has per-round probability 1/8 and rejection probability 1/4, so final probability $\sum_{r\geq 0}(1/4)^r(1/8) = 1/6$. Expected rounds are 4/3, giving four expected bits.

Direct mod 6 gives outputs 1 and 2 preimages {0,6} and {1,7}; each other output has one preimage. Probabilities are 1/4,1/4,1/8,1/8,1/8,1/8. Rejection sampling is exactly uniform and terminates almost surely because $(1/4)^r \to 0$, but has no fixed worst-case number of rounds.

05-E14 Fair bits from an unknown biased coin

Flip twice per round. HT returns 0, TH returns 1; HH and TT retry. Both accepted outcomes have probability p(1−p). The acceptance rate q=2p(1−p)>0 gives each final bit probability $\sum_{r\geq 0}(1-q)^r p(1-p) = 1/2$, and infinite retry probability zero.

Expected rounds are 1/q, hence expected flips $1/[p(1-p)]$. This uses independent flips with the same p; equal marginals alone do not provide the same pair structure. Every fixed 0<p<1 has finite expectation, but it is unbounded near 0 or 1, so no uniform O(1) bound holds over p. At the excluded endpoints, acceptance never occurs.

Probabilistic analysis solutions (continued)

Verification code appendix: probability_depth_checks.py

```
"""Exact finite probability models for Chapter 5 examples; no simulation claims."""
from fractions import Fraction as F
from itertools import product,permutations,combinations
from math import factorial,comb,ceil,log2
from collections import Counter
from functools import lru_cache

def harmonic(n):return sum((F(1,i) for i in range(1,n+1)),F(0))
def shuffle_path(a,choices,reverse=False):
    a=list(a)
    for i,j in zip(range(len(a)-1,0,-1) if reverse else
range(len(a)),choices):a[i],a[j]=a[j],a[i]
    return tuple(a)
def records(a):
    highest=-1;out=[]
    for i,v in enumerate(a):
        if v>highest:out.append(i);highest=v
    return out
def online(a,k):
    threshold=max(a[:k])
    return next((i for i in range(k,len(a)) if a[i]>threshold),len(a)-1)
def longest(a):
    best=current=0
    for v in a:
        current=current+1 if v else 0;best=max(best,current)
    return best

def check():
    shuffle_cases=0;hiring_cases=0;online_cases=0
    for n in range(9):
        counts=Counter(shuffle_path(range(n),js) for js in product(*(range(i,n) for i in
range(n))))
        assert len(counts)==factorial(n) and set(counts.values())==
{1};shuffle_cases+=sum(counts.values())
        rev=Counter(shuffle_path(range(n),js,True) for js in product(*(range(i+1) for i in
range(n-1,0,-1))))
        assert counts==rev
        hits=[0]*n;hist=Counter();wins=[0]*n
        for a in permutations(range(n)):
            rec=records(a)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Probabilistic analysis solutions (continued)

```
            for i in rec:hits[i]+=1
            hist[len(rec)]+=1;hiring_cases+=1
            for k in range(1,n):
                wins[k]+=a[online(a,k)]==n-1;online_cases+=1
        if n:
            assert sum(F(c,factorial(n)) for c in hits)==harmonic(n)
            assert all(F(c,factorial(n))==F(1,i+1) for i,c in enumerate(hits))
            assert hist[1]==factorial(n-1) and hist[n]==1
            if n>=2:assert F(hist[2],factorial(n))==harmonic(n-1)/n
        for k in range(1,n):assert F(wins[k],factorial(n))==F(k,n)*(harmonic(n-1)-harmonic(k-
1))
    assert online([2,5,1,4,6,3],2)==4
    assert online([3,1,6,2,4,7,5],2)==2
    assert max((F(k,7)*(harmonic(6)-harmonic(k-1)),k) for k in range(1,7))==(F(29,70),2)
    naive=Counter()
    for js in product(range(3),repeat=3):
        a=list(range(3))
        for i,j in enumerate(js):a[i],a[j]=a[j],a[i]
        naive[tuple(a)]+=1
    assert set(naive.values())=={4,5}
    priorities=Counter()
    for a,b in product(range(4),repeat=2):priorities[a<=b]+=1
    assert priorities=={True:10,False:6}
    birthday_cases=0
    for n in range(1,6):
        for k in range(6):
            collisions=0;pairs=0
            for a in product(range(n),repeat=k):
                collisions+=len(set(a))<k;pairs+=sum(a[i]==a[j] for i,j in
combinations(range(k),2));birthday_cases+=1
            total=n**k;distinct=F(1)
            if k<=n:
                for j in range(k):distinct*=F(n-j,n)
            else:distinct=F(0)
            assert F(collisions,total)==1-distinct
            assert F(pairs,total)==F(comb(k,2),n)
    q=F(1)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Probabilistic analysis solutions (continued)

```
    for j in range(5):q*=F(16-j,16)
    assert 1-q==F(4097,8192)
    ap=[(u,v,(2*u-v)%3) for u,v in product(range(3),repeat=2)]
    for i,j in combinations(range(3),2):assert len({(a[i],a[j]) for a in ap})==9
    assert sum(len(set(a))<3 for a in ap)==3
    weighted_collision=weighted_pairs=F(0);probs=[F(1,2),F(1,4),F(1,8),F(1,8)]
    for a in product(range(4),repeat=3):
        p=probs[a[0]]*probs[a[1]]*probs[a[2]]
        weighted_collision+=p*(len(set(a))<3);weighted_pairs+=p*sum(a[i]==a[j] for i,j in
combinations(range(3),2))
    assert weighted_collision==F(95,128) and weighted_pairs==F(33,32)
    balls=0
    for b in range(1,5):
        for t in range(6):
            empty=single=covered=0
            for a in product(range(b),repeat=t):
                loads=Counter(a);empty+=b-len(loads);single+=sum(v==1 for v in
loads.values());covered+=len(loads)==b;balls+=1
            assert F(empty,b**t)==b*F(b-1,b)**t
            expected_single=t*F(b-1,b)**(t-1) if t else F(0)
            assert F(single,b**t)==expected_single
            if (b,t)==(3,4):assert F(covered,b**t)==F(4,9)
        E=F(0)
        for i in range(b-1,-1,-1):E=(1+F(b-i,b)*E)/(1-F(i,b))
        assert E==b*harmonic(b)
    streak_cases=0
    for n in range(13):
        values=[];windows=[0]*(n+1)
        for a in product([0,1],repeat=n):
            L=longest(a);values.append(L);streak_cases+=1
            for k in range(1,n+1):windows[k]+=sum(all(a[i:i+k]) for i in range(n-k+1))
        for k in range(1,n+1):assert F(windows[k],2**n)==F(n-k+1,2**k)
        if n:
            E=F(sum(values),2**n)
            assert E==sum((F(sum(v>=k for v in values),2**n) for k in range(1,n+1)),F(0))
            assert E<=ceil(log2(n))+1
        if n==8:assert sum(v>=3 for v in values)==107
    seq=[c=='H' for c in 'HHHTHHHHTTHTHHTH']
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Probabilistic analysis solutions (continued)

```
    assert longest(seq)==4 and [i+1 for i in range(14) if all(seq[i:i+3])]==[1,5,6]
    @lru_cache(None)
    def trees(n):
        if n==1:return ((1,0,F(1)),)
        out=[]
        for p in range(1,n):
            for la,ca,pa in trees(p):
                for lb,cb,pb in trees(n-p):out.append((la+lb,n+ca+cb,pa*pb/F(n-1)))
        return tuple(out)
    tree_cases=0
    for n in range(1,10):
        cases=trees(n);assert sum(p for l,c,p in cases)==1
        assert all(l==n for l,c,p in cases)
        assert sum(c*p for l,c,p in cases)==2*n*(harmonic(n)-1);tree_cases+=len(cases)
    quick_cases=0
    for n in range(1,8):
        total=0
        for order in permutations(range(n)):
            priority={v:i for i,v in enumerate(order)}
            def cost(keys):
                if len(keys)<=1:return 0
                p=min(keys,key=priority.get)
                return len(keys)-1+cost([v for v in keys if v<p])+cost([v for v in keys if
v>p])
            total+=cost(list(range(n)));quick_cases+=1
        assert F(total,factorial(n))==2*(n+1)*harmonic(n)-4*n
    for m in range(1,34):
        k=(m-1).bit_length();accepted=[z for z in range(2**k) if z<m]
        assert len(accepted)==m and 2**k<2*m
    return
{'forward_and_reverse_shuffle_paths':shuffle_cases,'hiring_rank_permutations':hiring_cases,'o
nline_threshold_permutation_cases':online_cases,'birthday_assignments':birthday_cases,'indepe
ndent_ball_assignments':balls,'binary_streak_sequences':streak_cases,'random_split_weighted_t
rees':tree_cases,'uniform_pivot_priority_orders':quick_cases,'six_person_online_success_k2':'
77/180','seven_person_optimum':{'k':2,'success':'29/70'},'nonuniform_birthday_example':
{'collision_probability':'95/128','pair_expectation':'33/32'},'scope':'Exact finite models
validate traces and counterexamples; general probability/invariant/asymptotic proofs are in
the canonical chapter.'}

if __name__=='__main__':
    import json
    print(json.dumps(check(),ensure_ascii=False,indent=2))
```

Probabilistic analysis solutions (continued)

Verification code appendix: random_partition.py

```
"""Exact recurrence checks for random recursive splits and quicksort."""

from fractions import Fraction


def random_split_expectations(n):
    leaves = [Fraction(0)] * (n + 1)
    stars = [Fraction(0)] * (n + 1)
    leaves[1] = Fraction(1)
    for size in range(2, n + 1):
        leaves[size] = Fraction(2, size - 1) * sum(leaves[1:size])
        stars[size] = size + Fraction(2, size - 1) * sum(stars[1:size])
    return leaves, stars


def harmonic(n):
    return sum((Fraction(1, k) for k in range(1, n + 1)), Fraction(0))


def quicksort_comparisons(n):
    return sum(
        (Fraction(2, distance + 1) * (n - distance) for distance in range(1, n)),
        Fraction(0),
    )

def online_success(k, n):
    return Fraction(k, n) * sum(
        (Fraction(1, i) for i in range(k, n)), Fraction(0)
    )


def expected_longest_head_run(n):
    total = 0
    for bits in range(1 << n):
        longest = run = 0
        for position in range(n):
            if bits >> position & 1:
                run += 1
                longest = max(longest, run)
            else:
                run = 0
        total += longest
    return Fraction(total, 1 << n)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probabilistic analysis solutions (continued)

```
def main():
    maximum = 12
    leaves, stars = random_split_expectations(maximum)
    for n in range(1, maximum + 1):
        assert leaves[n] == n
        assert stars[n] == n * (2 * harmonic(n) - 2)

    rows = []
    for n in range(1, 9):
        rows.append(
            (
                n,
                leaves[n],
                stars[n],
                quicksort_comparisons(n),
            )
        )
    assert rows[4][1:] == (
        Fraction(5),
        Fraction(77, 6),
        Fraction(37, 5),
    )
    assert stars[2] == 2 and stars[3] == 5
    best_k = max(range(1, 100), key=lambda k: online_success(k, 100))
    assert online_success(best_k, 100) > Fraction(1, 3)
    streak_rows = [
        (n, expected_longest_head_run(n)) for n in range(1, 11)
    ]
    assert all(current >= previous for (_, current), (_, previous) in zip(streak_rows[1:],
streak_rows))
    print(
        "PASS random_split_rows="
        + repr([tuple(map(str, row)) for row in rows])
        + f" online_k_100={best_k}"
        + f" online_success_100={float(online_success(best_k, 100)):.6f}"
        + " longest_runs="
        + repr([(n, str(value)) for n, value in streak_rows])
    )


if __name__ == "__main__":
    main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study